

Design: Motion Vector Difference Matching (MVDM)

Data Preparation

- Extract the test data from TA's directory:
% tar xvf ~iclabTA01/Final_Project_2025F.tar

Design Description

Motion Vector Difference Matching is a technique used in video processing and compression. It analyzes the differences between motion vectors of consecutive frames to predict movement more accurately. By comparing these differences, the method helps reduce redundant information in video data, leading to more efficient compression and smoother playback. For example, like Fig 1.

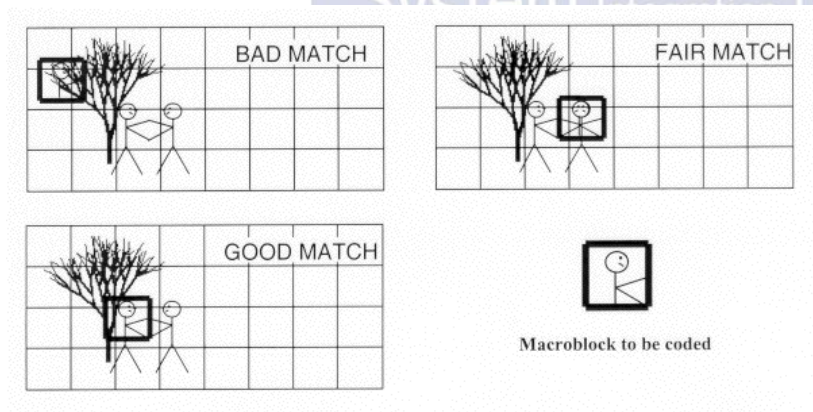


Fig 1. The Motion Vector Difference Matching example

There are lots of methods of calculating image similarity for finding the matching parts, such as “Sum of Absolute Differences”, “Cross Correlation”, “Cross Coefficient”, etc. In this exercise, you are asked to use the “Sum of Absolute Transformed Differences (SATD)” method, which is a metric used in computer science that calculates the total sum of absolute pixel value differences between two images.

Image processing is a method to convert an image into digital form and perform some operations on it, in order to get an enhanced image or to extract some useful information from it.

In this exercise, you need to build a design to perform motion vector difference matching and doing some image processing. For the matrix requires large space to store, you are asked to use memory(SRAM) for finishing this lab.

- Image (L0 and L1)

You will first receive the two images L0 and L1 by input bus “in_data”, and you will need to store it using SRAM for calculations later.

➤ Size (Row × Col)

The **image size** will be 128×128 for both image L0 and L1.

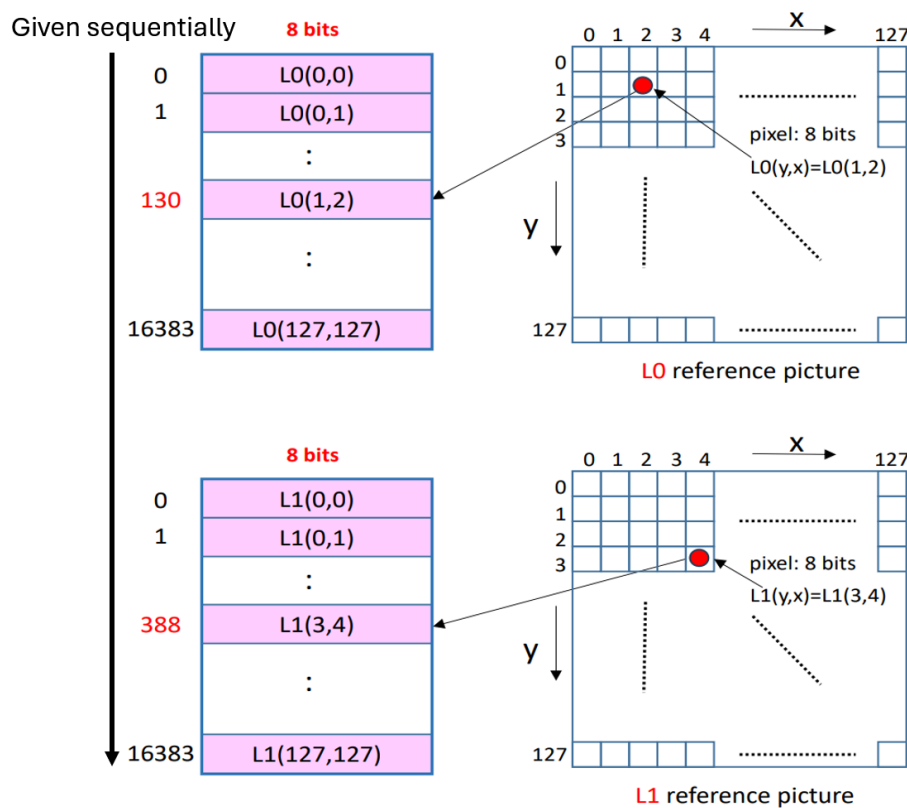


Fig 2. The data sending order of L0 and L1 reference picture

➤ Element

The element of an image pixel is an 8-bit unsigned integer.

	[8:1]	[0]
[8:0] in_data	Image_data	X (unknown)

Table 1. image pixel data input by “in_data”

● Calculate instruction

➤ MV (motion vector)

The MV provides the position(y, x) of L0 and L1 where it needs to be calculated, and MV also provides the fraction to calculate the Bilinear Interpolation(BI). You will receive two pairs of MV, each will be 4 cycle long by “in_data” input bus. Notice that the second pair of MV’s position point2(y, x) will be within the range of point1($y \pm 5$, $x \pm 5$) for both L0 and L1.

Cycle	in_data[8:1] MV integer	in_data[0] MV fraction
-------	-------------------------	------------------------

1	8 bit integer MVx of L0 of point1	frac_x of L0 of point1
2	8 bit integer MVy of L0 of point1	frac_y of L0 of point1
3	8 bit integer MVx of L1 of point1	frac_x of L1 of point1
4	8 bit integer MVy of L1 of point1	frac_y of L1 of point1
5	8 bit integer MVx of L0 of point2	frac_x of L0 of point2
6	8 bit integer MVy of L0 of point2	frac_y of L0 of point2
7	8 bit integer MVx of L1 of point2	frac_x of L1 of point2
8	8 bit integer MVy of L1 of point2	frac_y of L1 of point2

Table 2. The MV instruction input

Note: The maximum value of the MV integer signal is below 117.00

● Action

You are asked to perform 6-tap FIR Filter Interpolation for half-pixel precision generation, which is a standard technique in H.264/AVC.

➤ Interpolation (6-tap FIR Filter)

Instead of simple linear averaging, we use a 6-tap filter to generate the half-pixel value (0.5 position) from the 6 nearest integer pixels. Assume we are calculating the point P_0 's half-pixel, and here is the formula:

- **Coefficients:** [1, -5, 20, 20, -5, 1]
- **Formula:** $Val = (P_{-2} - 5P_{-1} + 20P_0 + 20P_1 - 5P_2 + P_3)$
- **Fraction Definition (1-bit):**
 - 0: Integer position (Copy pixel directly).
 - 1: Half-pixel position (Perform Interpolation).

➤ Interpolation Cases:

Based on the fractional parts (frac_x, frac_y), there are four processing modes:

1. **No Interpolation** (frac_x=0, frac_y=0):
 - Apply no filter.
 - **Output:** P_0

2. **Horizontal Interpolation Only** (frac_x=1, frac_y=0):

- Apply the filter horizontally on the row.
- **Output:** $\text{Clip}((\text{Val} + 16) \gg 5)$

3. **Vertical Interpolation Only** (frac_x=0, frac_y=1):

- Apply the filter vertically on the column.
- **Output:** $\text{Clip}((\text{Val} + 16) \gg 5)$

4. **2D Separable Interpolation** (frac_x=1, frac_y=1):

- **Step 1 (Horizontal):** Apply horizontal filter first. **Do NOT clip** here. The intermediate result requires larger bit-width.
- **Step 2 (Vertical):** Apply vertical filter on the intermediate results.
- **Step 3 (Final):** Normalize and clip.
- **Output:** $\text{Clip}((\text{Val} + 512) \gg 10)$

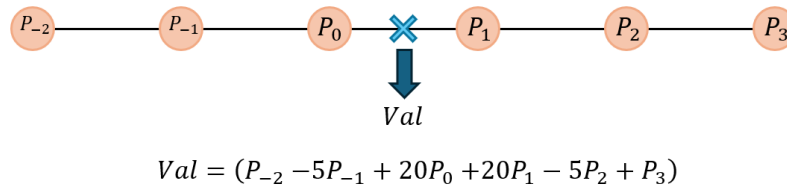


Fig 3. The calculation method of 6-tap FIR interpolation

Boundary Handling (Edge Replication)

Since the 6-tap FIR filter requires accessing pixels outside the current position (range $[x-2, x+3]$), access coordinates may fall outside the valid image boundary $[0, 127]$.

When a requested pixel coordinate P_x is unavailable, it must be replaced by the nearest valid pixel inside the boundary.

Mathematical Expression:

Let W be the width/height of the image (in this Lab, $W=128$). Let x be the coordinate index we want to access. The actual pixel value used for calculation, $P'(x)$, is defined as:

$$P'(x) = P(\text{Clip}(x))$$

Where the **Clip function** maps the input coordinate to the valid range $[0, W-1]$:

$$Clip(x) = \begin{cases} 0, & \text{if } x < 0 \\ x, & \text{if } 0 \leq x \leq W - 1 \\ W - 1, & \text{if } x > W - 1 \end{cases}$$

Example Case:

Suppose we need to calculate the horizontal interpolation at integer position $x=0$. The filter requires pixels at indices: $x-2, x-1, x, x+1, x+2, x+3 \Rightarrow -2, -1, 0, 1, 2, 3$

Since indices -2 and -1 are outside the left boundary (<0), they are replaced by the nearest valid pixel, which is index 0 .

- $P(-2) \rightarrow P(0)$
- $P(-1) \rightarrow P(0)$

The calculation formula becomes:

$$Val = 1P(0) - 5P(0) + 20P(0) + 20P(1) - 5P(2) + 1P(3)$$

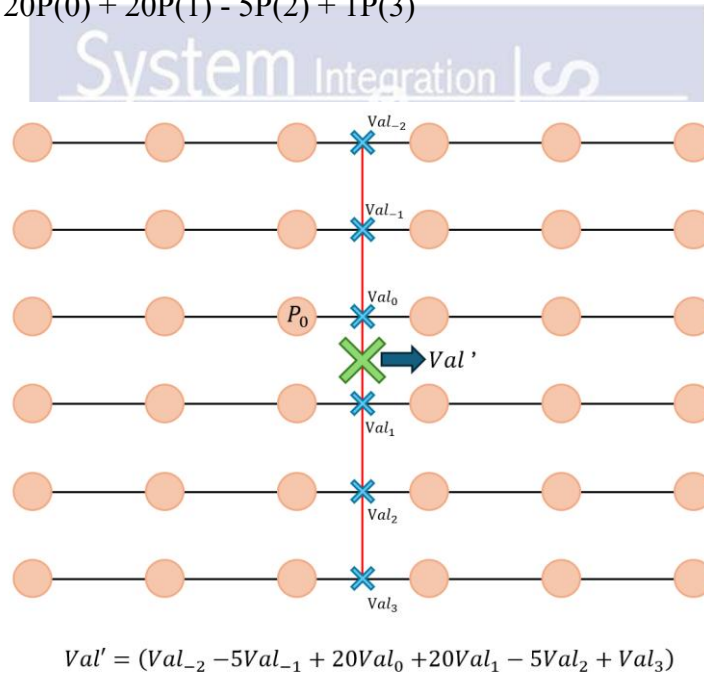


Fig 4. The calculation method when doing 2D Separable Interpolation ($frac_x=1, frac_y=1$)

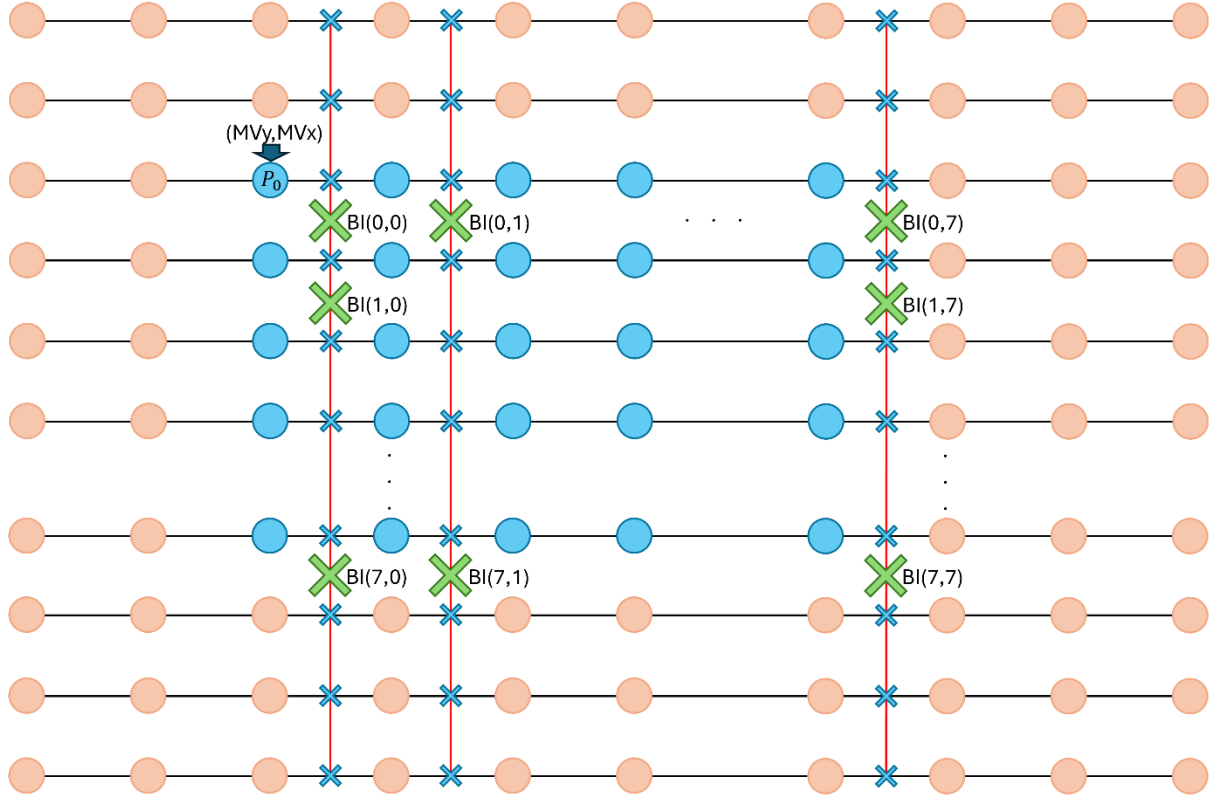


Fig 5. An example of the composition of BI_8x8 of L0(MVy, MVx)

➤ SATD (Sum of Absolute Transformed Difference)

After completing BI_8x8 calculation of L0/L1, start to perform Mirror MVD Matching algorithm shown as Figure 6, the search range is 3x3. The comparison must be carried out in the order of 0, 1, 2, ..., 8. The comparison method is to take BI_8x8 headed by “L0(MVy of L0, MVx of L0)” and BI_8x8 headed by “L1(MVy+2 of L1, MVx+2 of L1)” as the Sum of Absolute Transformed Difference(SATD) between the two matrices, this is the SATD value of “search Point = 0”. The same calculation method is extended to nine search points, which completes the Mirror MVD Matching.

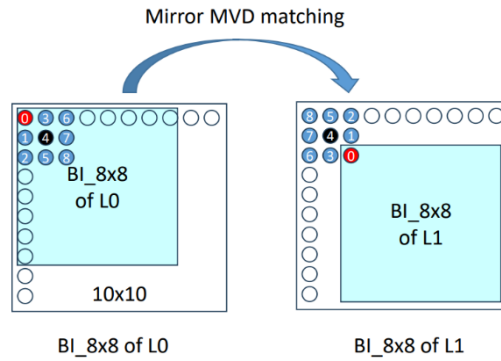


Fig 6. The search order of Mirror MVD Matching

you must implement **SATD**, which transforms the residual into the frequency domain using **Hadamard Transform** before summing up. This metric better reflects the visual quality.

Calculation Steps:

1. **Residual:** Calculate difference block $D = L0 - L1$.
2. **4x4 Partition:** Divide the 8X8 difference block into **four** 4X4 sub-blocks.
3. Hadamard Transform: For each 4X4 sub-block, calculate:

$$Y = H \times D_{4 \times 4} \times H^T$$

Where H is the 4x4 Hadamard matrix (consisting of 1 and -1).

4. Summation: Sum the absolute values of all coefficients in Y.

$$SATD_{4 \times 4} = \sum |Y_{ij}|$$

5. **Total Cost:** $SATD_{total} = \sum(\text{four } 4 \times 4 \text{ SATDs})$

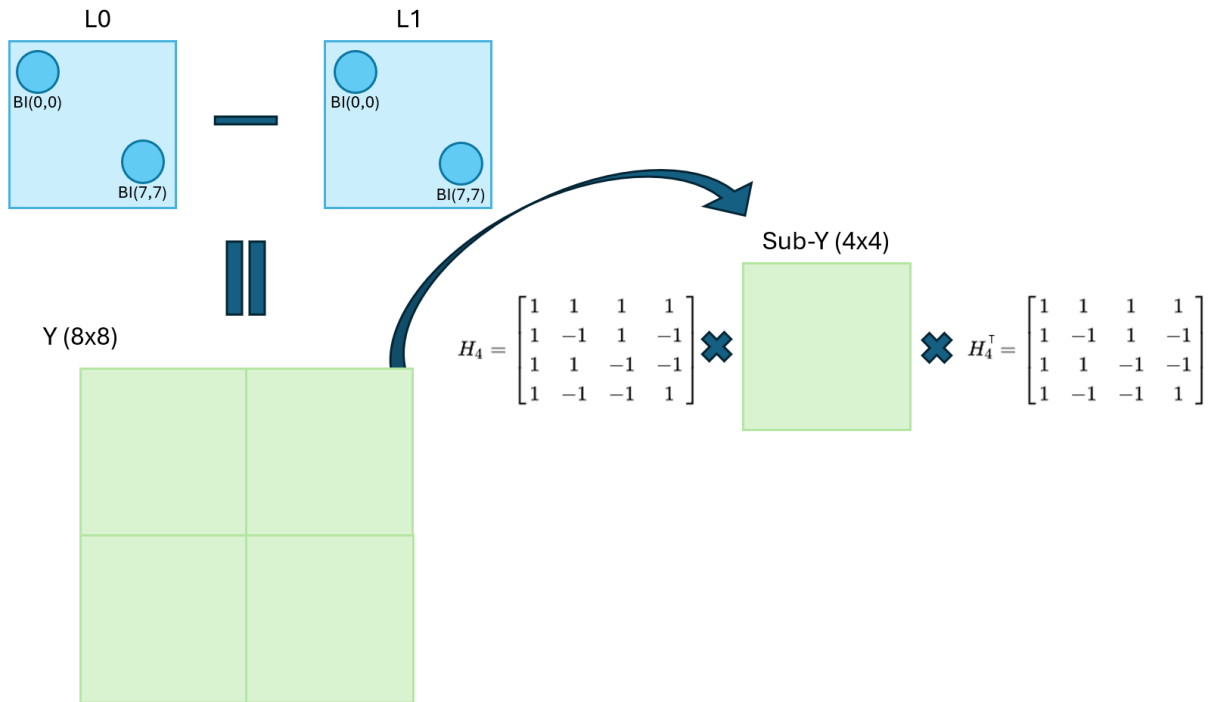


Fig 7. Example calculation method of SATD_8x8

● Output

After completing the calculation of SATD(0) ~ SATD(8), find the smallest SATD value and the search Point of smallest SATD from the nine SATD values, and then output them. When the SATD signal is to be output, the valid signal must be set to high to inform the Host that the current output data of the SATD pin is a valid output. Subsequent MV data are also processed in the same way. While all given MV data are output, the MVDM circuit design is completed.

However, in this lab, there are two sets of search point with different MV given. And the output pin will only be 1bit because we want to reduce the total pin number. So we need to send the output data from LSB to MSB like shown in figure 9.

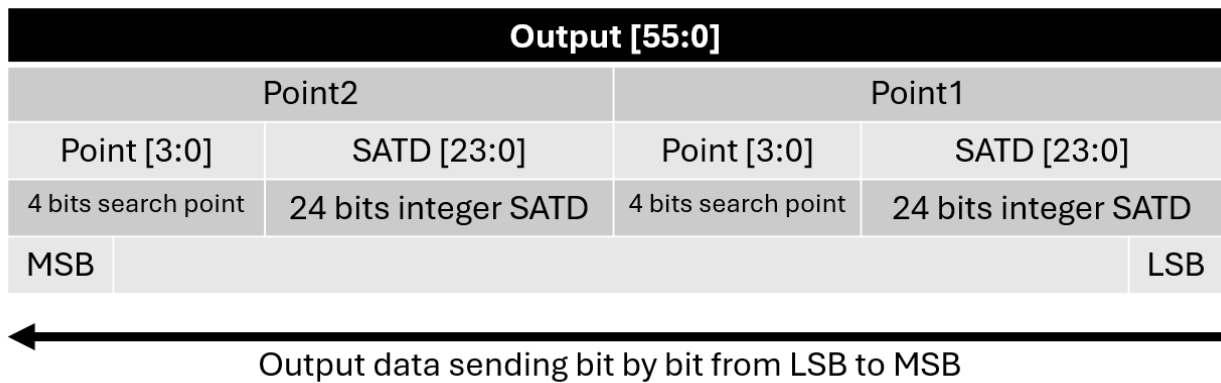


Fig 9. The output data format

After 64 sets of search points, you will get the next pattern(L0/L1).

- The flow for input/output

Flow	Pattern1	Set 1		Set 2		...	Set 64		Pattern2	Set 1		...
Behavior	Input	Input	Output	Input	Output	...	Input	Output	Input	Input	Output	...
Objects	-> L0 -> L1	MV*2set	SAD*2 (serial out)	MV*2set	SAD*2 (serial out)	...	MV*2set	SAD*2 (serial out)	-> L0 -> L1	MV*2set	SAD*2 (serial out)	...

Table 3. The input/output flow for this lab

I/O Description

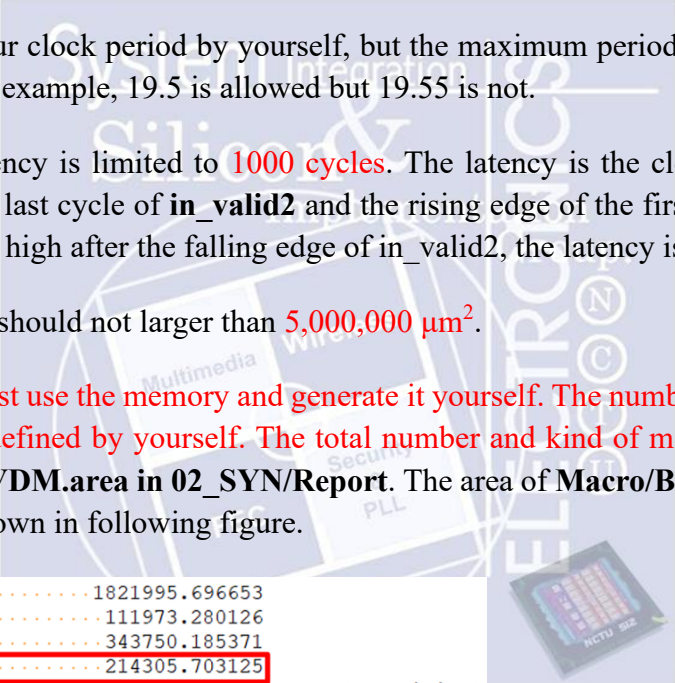
Input	Bit Width	Definition and Description
clk	1	Clock
rst_n	1	Asynchronous active-low reset
in_valid	1	High when "in_data" is sending L0/L1 values.
in_valid2	1	High when "in_data" is sending MV values.
in_data	9	Input data. When "in_valid" is high, in_data[8:1] is sending the data(8 bits unsigned integer) of L0 and L1, and in_data[0] remain x(unknown). When "in_valid2" is high, in_data[8:0] is sending the MV data, where it contains 8 bits of MV integer part and 1 bits of MV fraction part(please refer to table 2).

Output	Bit Width	Definition and Description
out_valid	1	High only when out_sad is valid. It cannot be overlapped with in_valid and in_valid2
out_sad	1	It will output the final result of the two sets of search point in serial out format from the LSB after finishing a set of caculation. Each out_sad contains 56 bits for two search point's lowest SAD value(please refer to figure 9).

Specification

Top module

1. Top module name: **MVDM** (file name: **MVDM.v**)
2. You can adjust your clock period by yourself, but the maximum period is **20 ns**. The precision of clock is 0.1, for example, 19.5 is allowed but 19.55 is not.
3. The execution latency is limited to **1000 cycles**. The latency is the clock cycles between the falling edge of the last cycle of **in_valid2** and the rising edge of the first cycle of **out_valid**. If the out_valid pulls high after the falling edge of in_valid2, the latency is counted as 1 cycle.
4. The total cell area should not larger than **5,000,000 μm^2** .
5. In this lab, you **must use the memory and generate it yourself**. The number of words and the bits per each word is defined by yourself. The total number and kind of memory is unlimited. We will check it at **MVDM.area** in **02_SYN/Report**. The area of **Macro/Black Box** must not be 0. The example is shown in following figure.



```

Combinational area:.....1821995.696653
Buf/Inv area:.....111973.280126
Noncombinational area:.....343750.185371
Macro/Black Box area:.....214305.703125
Net interconnect area: .....undefined (No wire load specified)

Total cell area:.....2380051.585150

```

6. There should be **no MEM_error allowed in the simulation**, and when having memory errors, you will fail this lab.

```

71  ** MEM_Error: Unknown value occurred (      118750 ps) in Address of TESTBED.u_MVDM.L0.ErrorMessage
72  ** MEM_Error: Unknown value occurred (      118750 ps) in Address of TESTBED.u_MVDM.L1.ErrorMessage

```

7. **If any port of memory is connected with mismatched width, the memory will not be synthesized and you will get an error message**. Even though the design may still pass gate level simulation, this situation will **be regarded as a synthesis fail**. In this case, the memory area will be 0 in **MVDM.area**. We will check it at **syn.log** and **MVDM.area**.

Reset

8. It is **asynchronous reset** and **active-low** architecture. If you use synchronous reset (reset after clock starting) in your design, you may fail to reset signals that should be reset after the reset signal is asserted.
9. The reset signal (**rst_n**) will be given only once at the beginning of simulation. **All output signals should be reset to 0 after the reset signal is asserted.**

in_valid & input signals

10. All input signals are synchronized at negative edge of the clock.
11. **in_valid** will come after reset.
12. **All the input data is sent by in_data for both L0/L1 and MV signals.**
13. The input of **image L0 and L1** is delivered in **raster scan order** for **128×128×2 cycles continuously** when **in_valid** is tied high. **When all elements of the two images are delivered completely, it is tied to unknown state**, and **in_valid** will also be tied low.
14. The input of **MV** is delivered in **sequence for 8 cycles continuously** when **in_valid2** is tied high. **When all elements of a MV are delivered completely, it will be tied to unknown state.**
15. **in_valid2** will be triggered for **a total 64 times** after **in_valid** falls in a single pattern. After each time **in_valid2** triggers, your design will do metrics matching with image processing and then **out_valid** will be tied high for corresponding cycles.
16. In each pattern, the first **in_valid2** will come in **3~6 negative edge of clock** after **in_valid** falls, and the other 63 times **in_valid2** will come in **3~6 negative edge of clock** after **out_valid** falls.

out_valid & out_value

All output signals should be synchronized at clock positive edge.

The TA's pattern will capture your output for checking at **clock negative edge**.

out_valid cannot overlap with **in_valid** and **in_valid2** at any time.

17. **out_valid** is limited to be high only when **out_sad** is valid.
18. **out_sad** should be correct when **out_valid** is high. And **out_sad** should be low when **out_valid** is low.

out_sad must be delivered for **total 56 cycles**, and **out_valid** should be high simultaneously.

The next input pattern will come in **3~6 negative edge of clock** after the **out_valid** of this pattern falls.

Synthesis

19. The input delay is set to **0.5*(clock period)**.
20. The output delay is set to **0.5*(clock period)**, and the output loading is set to **0.05**.
21. The input delay of **clk** and **rst_n** should be zero.
22. The synthesis time should be less than **3 hour**.
23. **Do not write comment in the filelist.f file**, or the simulation may pass in your terminal but **will FAIL** in TA's.

```
1 |TESTBED.v
2 |//../04_MEM/L0_16index_ver_1024X128.v
3 |//../04_MEM/L0_index8_2048X64.v
4 |../04_MEM/L0_index16_1024X128_SP.v
5 |../04_MEM/L1_index16_1024X128_SP.v
```

(NEVER do that)

24. The synthesis result (syn.log) of data type cannot include any **latches** and **error**.
25. After synthesis, you can check **MVDM.area** and **MVDM.timing**. The area report is valid when the slack in the end of timing report should be **non-negative** and the result should be **MET**.

Gate-level simulation

26. The gate-level simulation cannot include any timing violations **without the *notimingcheck* command**.

APR

27. Floorplanning

- Core size: Defined by user
- Core to IO boundary: Each side must be more than **100**
- Hard Macro placement: All hard macro should be **in CORE**

28. Powerplanning

- Core Ring
 - (i) Top & Bottom: metal layer must be **odd** (1,3,...) and width is **9**
 - (ii) Left & Right: metal layer must be **even** (2,4,...) and width is **9**
 - (iii) Each side must be **wire group**, **interleaving**, and at least **5 pairs**.

- Stripes
 - (i) Horizontal: metal layer must be **odd** (1,3,...) and width is at least **2**
 - (ii) Vertical: metal layer must be **even** (2,4,...) and width is at least **2**
 - (iii) The maximum distance between two stripes or stripe and edge should be less than **200**

29. Timing analysis results

- Timing slack: **No negative slacks** after setup/hold time analysis (including SI)
- Design Rule Violation (DRV): The **DRV of (fanout, cap, tran)** should be **all 0** after post-Route setup/hold time analysis (including SI)

30. Design verification results

- Layout v.s. Schematic (LVS): **NO LVS violations** after “verify Connectivity”
- Design Rule Check (DRC): **NO DRC violations** after “verify DRC”

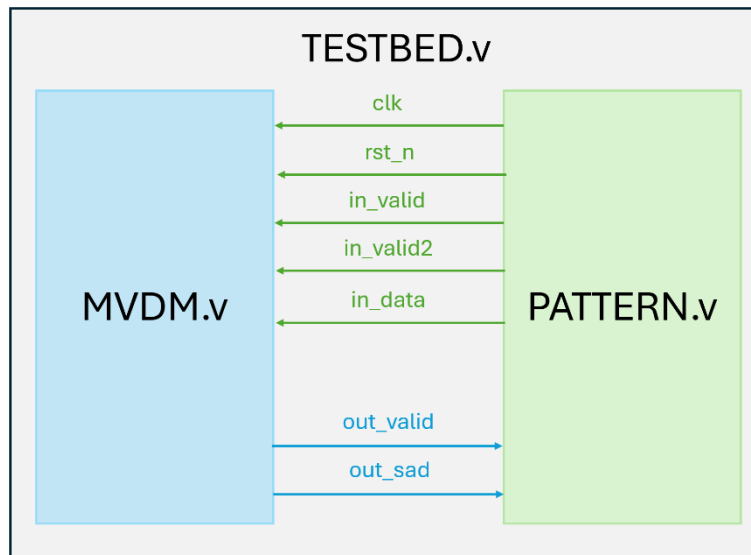
Post-simulation

31. The post-simulation cannot include any timing violations **without the *notimingcheck* command.**

Supplement

32. In this lab, you are **NOT** allowed to use Designware IP.
33. Don't use any wire/reg/submodule/parameter name called ***error***, ***congratulation***, ***latch*** or ***fail*** otherwise you will fail the lab. Note: * means any char in front of or behind the word. e.g: error_note is forbidden.
34. **Don't write Chinese or other language comments** in the file you sent.
35. Verilog command `//synopsys dc_script_begin, //synopsys dc_script_end //synopsys translate_off, //synopsys translate_on` are only allowed during the usage of including and setting designware IPs, other design compiler optimizations are forbidden.
36. Using the above commands are allowed, however any error messages during synthesis and simulation, regardless of the result will lead to failure in this lab.
37. Any form of **display or printing information in verilog design is forbidden.** You may use this methodology during debugging, but the file you turn in should not contain any coding that is not synthesizable.

Block Diagram



Grading Policy

The performance is determined by the area and latency of your chip. The less cost your design has, the higher grade you get.

Use your own design

- RTL/Gate-level/Post-level simulation, Synthesis and APR correctness: 70%
- Performance: 30%
 - **Chip area² * Total latency**
 - Total latency = cycle time * latency

Note

- Please submit your design in Final_Project/09_SUBMIT
 - 1st_demo deadline : **2025/12/22 (Mon.) 12:00:00**
 - 2nd_demo deadline: **2025/12/24 (Wed.) 12:00:00** (score will be 30% off)
- Please upload the following files under 09_SUBMIT
 - RTL design: **MVDM.v**
 - filelist: **filelist.f**
 - Memory file: **all your memory files .v / .db / .lef / .lib in 04_MEM**
 - APR file: **CHIP.sdc / CHIP.inn / CHIP.io / CHIP.v / CHIP.sdf / CHIP.inn.dat**

➤ In this lab, you can adjust your clock cycle time. Consequently, make sure to key in your clock cycle time after the command like the figure below. It's means that the TA will demo your design under this clock cycle time.

➤ If your files **violate the naming rule**, you will get **10 deduction points**.

● Template folders and reference commands:

➤ 01_RTL/ (RTL simulation) **./01_run_vcs_rtl**

➤ 02_SYN/ (Synthesis) **./01_run_dc_shell**

■ Check if there is any latch in your design in syn.log

■ Check the timing of design in /Report/MVDM.timing

■ Check the area of design in /Report/MVDM.area

➤ 03_GATE/ (Gate-level simulation) **./01_run_vcs_gate**

➤ 04_MEM/ (Memory location)

■ **Your should generate your own memory and put the required files here**

➤ 05_APR/ (APR Folder) **./00_combine**

➤ 06_POST/ (Post-level simulation) **./01_run_vcs_post**

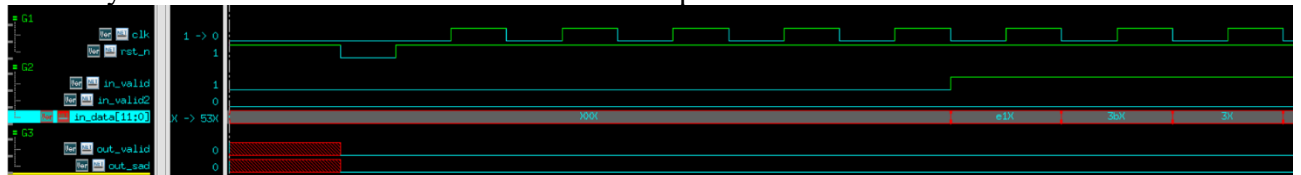
➤ 09_SUBMIT/ (Submit your files) **./00_tar**

./01_submit

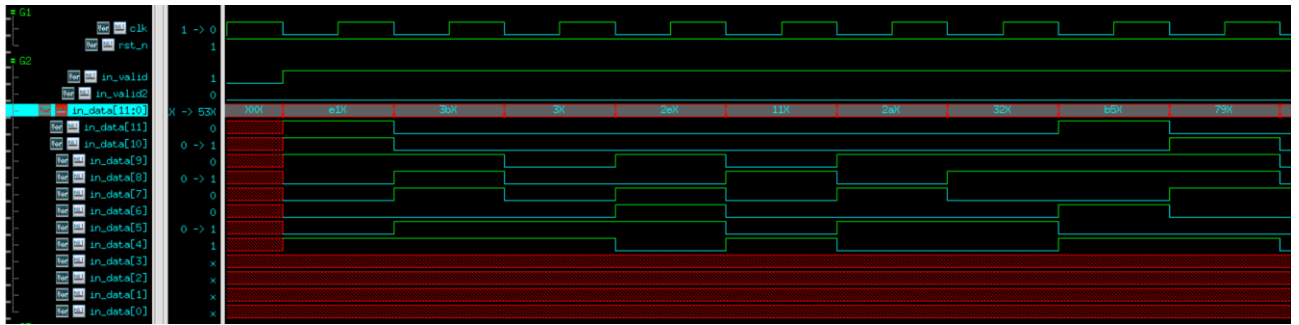
./02_check

Reference Waveform

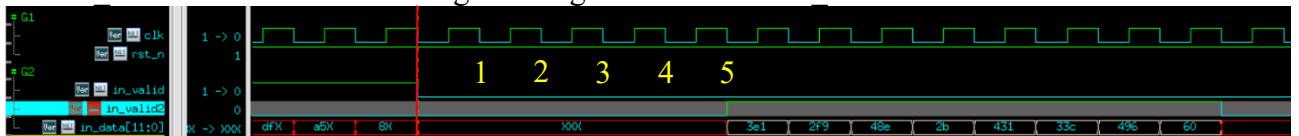
● Asynchronous reset and active-low reset all output to low



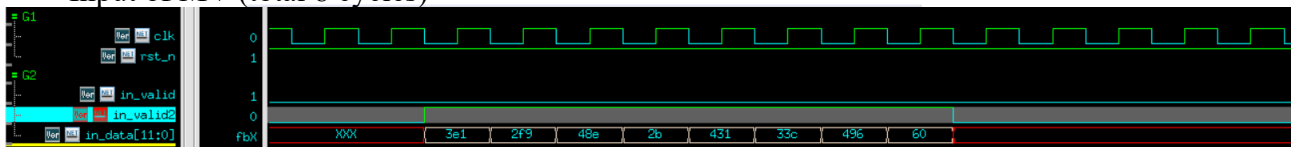
● Input of L0/L1 when in_valid is high(total 128*128*2 cycles)



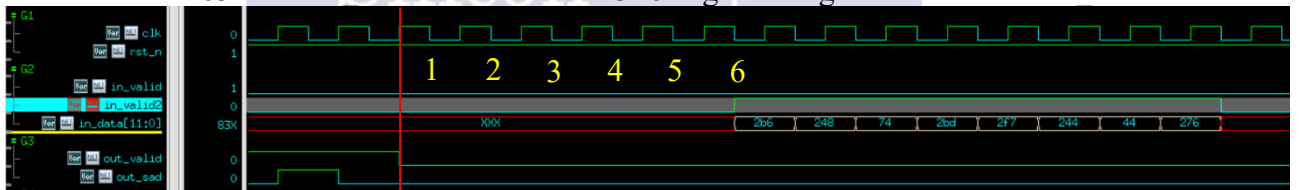
- in_valid2 will come in 3~6 negative edge of clock after in_valid is tied low



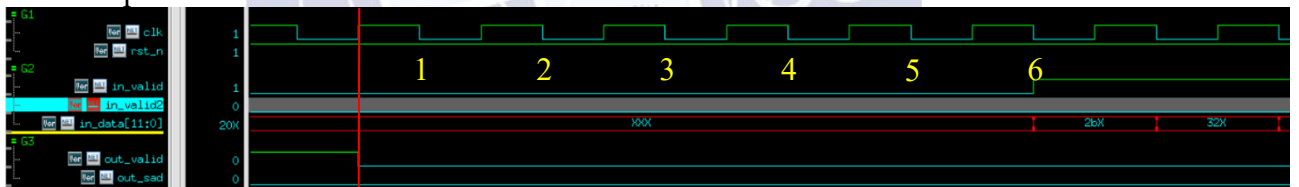
- Input of MV (total 8 cycles)



- The other 63 times in_valid2 will come in 3~6 negative edge of clock after out_valid falls



- The next input pattern will come in 3~6 negative edge of clock after the 64th out_valid of this pattern falls



- The definition of latency

